SuperSploit Framework

Architecture & Command Workflow Specification

SuperSploit is a lightweight, Python 3-based exploitation, reconnaissance, and payload management framework. Designed as an agile alternative to heavier platforms like Metasploit, it provides a centralized execution hub for interactive penetration testing, dynamic payload compilation, and hardware-level attacks. This document details its newly refactored, enterprise-grade architecture.

1. Core Architectural Domains

The framework is divided into four highly decoupled, modular domains, ensuring scalability and strict security boundaries.

A. Core Engine (CLI & Routing)

The core engine manages the user interface, session state, and command execution routing.

- **Interactive Terminal:** Utilizes `prompt_toolkit` to provide a fluid, interactive shell complete with persistent command history, auto-suggestion, and ANSI-colored output.
- **O(1) Command Registry:** Commands are routed using an optimized dictionary mapping rather than linear parallel lists. This allows the framework to instantly dispatch inputs (e.g., `recon-ng`, `port-scan`) to their respective function handlers with maximum efficiency.
- **Input Sanitization:** Before any command hits the underlying OS, arguments are strictly parsed using `shlex.split()`. This guarantees that shell metacharacters (quotes, ampersands, semicolons) are safely sanitized, completely mitigating arbitrary command injection vulnerabilities.

B. Exploit Handler

The Exploit Handler dynamically processes local and remote payloads stored in the framework's payload database.

- **Safe Dynamic Python Execution:** Instead of writing Python payloads directly into the framework's source tree (which causes cache poisoning), the framework utilizes `importlib.util`. Exploits are loaded into isolated memory namespaces and executed safely.
- **On-the-Fly C Compilation:** The framework detects `.c` payloads, prompts the user for specific `gcc` compiler flags, compiles the binary to a temporary file, executes it interactively, and guarantees cleanup via a `finally` block, even if the payload crashes.
- **Bash Script Wrapper:** Executes raw shell scripts while securely capturing exit codes for error logging.

C. Hybrid Security Validation Layer

> **Zero-Trust Execution Model:** SuperSploit implements a hybrid integrity verification system to ensure no external binary or script is executed if it has been tampered with.

- **System Packages (`dpkg`):** For tools installed via the OS package manager (like `nmap`, `bettercap`), the framework queries `dpkg -V`. This allows the OS to seamlessly update these tools without breaking framework checksums.
- **Custom Scripts (`hashlib`):** For downloaded binaries or internal framework scripts (like `BlueDucky.py`), the framework uses native Python SHA256 hashing to compare the file against a trusted `checksums.json` registry.

D. Reconnaissance & Bluetooth Cores

A specialized suite of automated intelligence gathering and hardware attack vectors.

- **Nmap Wrapper:** Automates ping sweeps (`-sn`) and OS detection (`-O`), parsing the output into a structured `targets.json` tracking file.
- **OSINT Tools:** Integrates `phoneinfoga` for phone number footprinting and `namesearch.py` for generating Google Dorks across social media platforms.
- **BlueDucky Subsystem:** Transforms the host machine into a malicious Bluetooth Human Interface Device (HID). It establishes direct L2CAP connections (bypassing

standard pairing protocols) to inject raw hexadecimal keyboard reports, processing complex `DuckyScript` payloads with custom Shift-key mapping.

2. Command Workflow Trace

The following illustrates the exact lifecycle of a user command as it travels through the SuperSploit architecture. For this example, we trace the command: `scan-target -p 80,443 -sV`

Step 1: Input Ingestion & Routing

The user types the command into the `prompt_toolkit` session. The `inputHandler.py` receives the raw string. It identifies the first word (`scan-target`) and performs an O(1) dictionary lookup against the `wifi_cmds` registry, routing the request to `nmapWrapper.targeted_scan()`.

Step 2: Interactive Prompts

The `targeted_scan()` function queries the active `targetlist` database. It safely displays an enumerated list of targets. The user enters an index (e.g., `1`). A `try/except (ValueError, IndexError)` block ensures the framework does not crash if the user enters invalid characters.

Step 3: Security & Integrity Verification

Before launching the scanner, the framework calls `validator.verify_system_package("nmap")`. The `security.py` module executes `dpkg -V nmap`. If the binary is intact, it returns `True`. If tampered with, it halts execution and warns the user.

Step 4: Sanitization & Construction

The raw arguments (`-p 80,443 -sV`) are passed through `shlex.split()` to guarantee no malicious shell injection is present. The command is securely assembled as a Python list: `['sudo', 'nmap', '-p', '80,443', '-sV', '192.168.1.5']`.

Step 5: Execution & Output Parsing

The list is passed to `subprocess.Popen`. Output is piped back to the SuperSploit terminal. Once completed, the `osdetect` function parses the stdout to extract newly discovered hostnames or device vendors (like Apple devices) and updates the persistent `targets.json` tracking file.